סיכום מקיף: עקרונות תכנות מונחה עצמים (OOP)

1. הרשאות גישה (Access Modifiers)

ב-Java ישנן 4 רמות חשיפה הקובעות מי יכול לגשת למשתנים ולמתודות שלנו.

הרשאה(Modifier)	מילה שמורה	למי יש גישה?
פרטי	private	רק המחלקה עצמה.
ברירת מחדל	(ריק)	המחלקה + מחלקות אחרות באותו.Package
מוגן	protected	אותו + Package מחלקות יורשות (Subclasses) בכל מקום.
ציבורי	public	כולם (כל המחלקות בכל הפרויקט).

כלל אצבע: נשתדל תמיד להיות הכי סגורים שאפשר (private), ונפתח (public/protected) רק כשיש סיבה מוצדקת.

---

2. סוגי קשרים בין מחלקות

ירושה (Inheritance) - extends

- **הגדרה:** קשר של "Is-A" (הוא סוג של...). משמש כאשר יש מכנה משותף ברור למשפחה של מחלקות.
- **הכלל:** ב-Java, מחלקה יכולה לרשת ממחלקה אחת בלבד.
- **בנאים:** בנאים לא עוברים בירושה. חובה ליצור בנאי בבן ולקרוא לאבא באמצעות ()super.

ממשק (Interface) - implements

- **הגדרה:** קשר של "Can-Do" (יכולת / חוזה). מגדיר התנהגות ללא קשר למשפחה הביולוגית.
- **הכלל:** מחלקה יכולה לממש אינסוף ממשקים.
- **משתנים בממשק:** כל משתנה בממשק הוא אוטומטית public static final. כלומר: הוא קבוע (Constant) וגלובלי. לא ניתן להגדיר משתנים רגילים.

---

3. השוואה: מחלקה אבסטרקטית מול ממשק

תכונה	מחלקה אבסטרקטית (Abstract Class)	ממשק (Interface)
מהות	זהות ("מי אתה")	יכולת ("מה אתה יודע לעשות")
יצירת מופע (new)	לא ניתן	לא ניתן
שדות	רגילים(private, protected...)	קבועים בלבד (static final)
בנאים	יש (לאתחול שדות)	אין
שימוש	שיתוף קוד בסיסי בין יורשים	יצירת סטנדרט אחיד למחלקות שונות

---

4. פולימורפיזם (Polymorphism)

היכולת להתייחס לאובייקטים מסוגים שונים בצורה אחידה דרך מצביע מטיפוס האב/הממשק.

א. העמסה (Overloading) - זמן קומפילציה

יצירת מתודות עם אותו שם באותה מחלקה, אך עם פרמטרים שונים (סוג או כמות).

- **חוקים:**
  - חובה לשנות את רשימת הפרמטרים.
  - שינוי הערך המוחזר בלבד אינו נחשב העמסה ויגרום לשגיאה.
  - מותר לשנות הרשאות גישה (Public/Private) בחופשיות.

ב. דריסה (Overriding) - זמן ריצה

כתיבה מחדש של מתודה במחלקה הבן (Child) שכבר קיימת במחלקה האב (Parent).

- **חוקים:**
  - החתימה (שם ופרמטרים) חייבת להיות זהה לחלוטין.
  - הרשאות גישה: אסור לצמצם (אם האב public, הבן לא יכול להיות private). מותר להרחיב.
  - ערך מוחזר: חייב להיות זהה לזה של האב, או מחלקה יורשת שלו (Covariant Type).
  - מומלץ להשתמש באנוטציה @Override.

---

5. המרות טיפוסים (Casting)

כאשר משתמשים בפולימורפיזם (למשל Animal a = new Dog()), אנו מאבדים גישה לתכונות הייחודיות של הבן.

- **Upcasting (כלפי מעלה):**
  - המרה מאובייקט ספציפי לטיפוס הכללי.
  - קורה אוטומטית (Animal a = new Dog()).
  - בטוח תמיד.
- **Downcasting (כלפי מטה):**
  - המרה מהטיפוס הכללי חזרה לטיפוס הספציפי.
  - דורש המרה מפורשת: Dog d = (Dog) a.
  - סכנה: אם האובייקט אינו מהסוג הנכון, נקבל ClassCastException.
  - פתרון: חובה לבצע בדיקה מקדימה:

למשל

```
If (a instanceof Dog) {
    Dog d = (Dog) a;
}
```

---

6. דגשים ומוקשים נפוצים (Pitfalls)

1. הבנאי הדיפולטיבי (Default Constructor):

לכל מחלקה יש בנאי ריק שנוצר אוטומטית. ברגע שיצרתם בנאי כלשהו עם פרמטרים – הבנאי הריק נמחק.

- ***הבעיה:*** מחלקה יורשת מנסה לקרוא אוטומטית ל-()super (הריק). אם הוא נמחק, הקוד לא יתקמפל.
- ***הפתרון:*** להוסיף בנאי ריק באבא ידנית, או לקרוא מהבן ל-(super(params.

2. קריאה ל-super:

בתוך בנאי, הקריאה לבנאי של האבא (()super) חייבת להיות השורה הראשונה בבלוק.

3. מערך של מחלקה אבסטרקטית:

אי אפשר לעשות new למחלקה אבסטרקטית (()new Animal – שגיאה), אבל אפשר ליצור מערך מהסוג שלה: Animal[] zoo. המערך יכיל בפועל אובייקטים "אמיתיים" (כלבים, חתולים).

4. גישה ל-Private בירושה:

ירושה לא מעניקה לבן גישה לשדות private של האבא. לשם כך יש להשתמש ב-protected או ב-Getters/Setters.